

Data Governance Copilot

Interview Question Bank

Topic 07 — Airflow Ingestion Pipeline

12 questions · 4 sections · DAG Design · FileSensor · S3 · MWAA · Neon Metadata DB · Idempotency

Foundational Core concepts **Intermediate** Design decisions **Advanced** Deep internals **Gotcha** Real bugs / traps

DAG Architecture & Design Decisions

Q01 Foundational

Your project has four Airflow DAGs. Walk through each one — what it does, its schedule, and why it exists as a separate DAG rather than being combined.

Hint: Each DAG has a distinct trigger type and responsibility — explain the separation of concerns.

Key points to cover:

- `file_ingestion_dag`: `schedule=None`, triggered by `FileSensor` watching `AIRFLOW_DOCS_PATH` (`docs/` folder) — ingests new files dropped locally
- `on_demand_ingest_dag`: `schedule=None`, triggered via Airflow REST API from `POST /ingest` or MCP `ingest_document_url` tool — user-initiated ingestion of a specific file
- `collibra_sync_dag`: `schedule='0 6 * * *'` (daily 06:00 UTC) — syncs Collibra catalog metadata to keep local cache fresh
- `nightly_refresh_dag`: `schedule='0 2 * * *'` (nightly 02:00 UTC) — rebuilds stale embeddings, prunes old checkpoints, refreshes data product lists
- Why separate: each DAG has a different trigger mechanism (sensor vs API vs cron) — combining them forces all into a cron schedule
- Separation also enables independent failure handling: `collibra_sync` failure doesn't block document ingestion
- Operational benefit: each DAG can be paused, backfilled, or retried independently in the Airflow UI

Q02 Foundational

Why is `schedule=None` used for `file_ingestion_dag` and `on_demand_ingest_dag`? What is the difference between a sensor-triggered DAG and a schedule-triggered DAG?

Hint: Think about what triggers these DAGs and why cron scheduling would be wrong.

Key points to cover:

- `schedule=None` means the DAG never runs automatically on a time schedule — it only runs when explicitly triggered
- `file_ingestion_dag` is triggered by `FileSensor` detecting a new file in `docs/` — document arrival is event-driven, not time-driven
- `on_demand_ingest_dag` is triggered by Airflow REST API (`POST /api/v1/dags/{dag_id}/dagRuns`) — user-initiated, happens when user uploads via FastAPI or MCP
- Cron schedule would be wrong for ingestion: polling every 5 minutes wastes resources when no new files exist; misses files dropped between polls
- `FileSensor` is push-like: Airflow worker polls the filesystem path on a `poke_interval` — much more responsive than a 5-min DAG schedule
- `on_demand` pattern enables real-time ingestion: user uploads PDF → FastAPI calls Airflow REST → DAG runs within seconds
- `schedule=None` is also correct for HITL-style DAGs where external human action determines when the pipeline runs

```
# Airflow REST API trigger (from scripts/trigger_ingest.sh) curl -X POST
http://airflow:8080/api/v1/dags/on_demand_ingest_dag/dagRuns \
-H 'Content-Type: application/json' \
-u admin:admin \
-d '{"conf": {"file_path": "s3://bucket/docs/policy_v4.pdf"}}'
```

Q03**Intermediate**

Explain the FileSensor in file_ingestion_dag. How does it work, what are its failure modes, and how do you configure poke_interval and timeout?

Hint: FileSensor is a blocking operator — understand its resource implications.

Key points to cover:

- FileSensor polls a filesystem path on poke_interval (e.g. every 30 seconds) until the file/pattern exists or timeout is reached
- When file detected: sensor succeeds, downstream tasks execute (load → chunk → embed → upsert)
- Failure mode 1: timeout — if no file appears within timeout seconds, sensor raises AirflowSensorTimeout — DAG run fails
- Failure mode 2: file appears then disappears — sensor detects at poke time but downstream task can't read it (race condition with the writer)
- Failure mode 3: stale file — FileSensor re-detects the same file on every DAG run; need to move/archive after processing
- poke_interval=30s, timeout=3600s (1 hour) is a sensible default for document ingestion — not time-critical
- mode='reschedule': sensor releases the worker slot between pokes (not blocking a worker thread) — critical for production with limited workers
- Post-processing: after successful ingestion, move file to docs/processed/ or delete — prevents re-ingestion on next DAG run

```
# FileSensor configuration from airflow.sensors.filesystem import FileSensor wait_for_file =
FileSensor( task_id='wait_for_document', filepath=AIRFLOW_DOCS_PATH + '/*.pdf', poke_interval=30, #
check every 30s timeout=3600, # fail after 1 hour mode='reschedule', # release worker between pokes
dag=dag, )
```

S3 vs Local Storage & Dev/Prod Switching

Q04**Intermediate**

Your project uses local docs/ folder in dev and S3 in production. How does this switching work and what code changes are needed?

Hint: Think about the abstraction layer between Airflow tasks and the file storage backend.

Key points to cover:

- Dev: AIRFLOW_DOCS_PATH=./docs — FileSensor watches local filesystem; loaders.py reads local paths directly
- Prod: AIRFLOW_DOCS_PATH=s3://bucket/docs — FileSensor uses S3KeySensor (not FileSensor) to detect new S3 objects
- S3KeySensor watches for new objects under an S3 prefix — replaces FileSensor in the production DAG definition
- loaders.py in prod: download S3 object to a temp file using boto3 (s3.download_file), then load from temp path
- Cleaner abstraction: a storage_backend abstraction class with local and S3 implementations — DAG tasks call storage.read(path) regardless of backend
- POST /ingest endpoint uploads file to S3 (boto3 put_object) before triggering the Airflow DAG with the S3 path in conf
- Environment variable STORAGE_BACKEND=local|s3 drives the switching — same pattern as ENABLE MOCK
- MCP ingest_document_url: downloads URL to temp file → uploads to S3 → triggers DAG — same prod pipeline

Q05**Intermediate**

Why does your project use Neon PostgreSQL as the Airflow metadata DB rather than a local SQLite or separate RDS instance?

Hint: Think about operational simplicity and the direct URL requirement.

Key points to cover:

- Airflow metadata DB stores DAG definitions, task states, run history, variables, connections — must be persistent and shared across scheduler + webserver
- SQLite: not supported for production Airflow — no concurrent writes, scheduler and webserver would corrupt the DB
- Separate RDS: operationally correct but adds another managed DB to provision, monitor, and pay for
- Neon choice: already provisioned for the application — reuses the same Neon account, zero additional infrastructure
- Airflow uses `AIRFLOW_DB_URL` (direct URL, port 5432) not the pooled pgBouncer URL — Airflow's SQLAlchemy engine needs session-mode connections
- `AIRFLOW__DATABASE__SQLALCHEMY_CONN = ${AIRFLOW_DB_URL:-${DATABASE_URL}}` — falls back to main `DATABASE_URL` if separate URL not provided
- Separation concern: Airflow metadata and application data (checkpoints, embeddings) share the same Neon instance but different schemas/tables — no functional conflict
- MWAA migration replaces this: MWAA manages its own metadata DB (Aurora PostgreSQL) — `AIRFLOW_DB_URL` becomes irrelevant in MWAA

Q06**Advanced**

The `on_demand_ingest_dag` receives a `file_path` in its conf dict. What is the full task chain inside this DAG and how does it handle failures mid-pipeline?

Hint: Walk through every Airflow task and explain retry + failure isolation.

Key points to cover:

- Task 1 — `validate_input`: reads `conf['file_path']`, checks file exists and is a supported type (PDF/DOCX/PPTX/Excel/text) — raises `ValueError` if invalid
- Task 2 — `load_document`: calls `loaders.load_document(file_path)` → returns `List[Document]` — stored as XCom for downstream tasks
- Task 3 — `chunk_document`: calls `chunker.chunk_document(docs)` → `List[Chunk]` with metadata — XCom push
- Task 4 — `embed_chunks`: calls `embedder.embed(chunks)` in batches of 100 — most expensive step, can fail on OpenAI rate limits
- Task 5 — `upsert_vectors`: calls `store.upsert(embedded_chunks)` to Neon pgvector — SHA-256 dedup prevents duplicates
- Task 6 — `notify`: logs ingestion summary (chunks ingested, deduped, source file) — could send Slack/Teams notification
- Failure handling: each task has `retries=2`, `retry_delay=timedelta(minutes=1)` — transient failures (OpenAI 429, Neon timeout) auto-recover
- Mid-pipeline failure: Airflow marks failed task red; upstream task outputs (XCom) are preserved — can clear and re-run from failed task
- Idempotency: SHA-256 dedup in upsert means re-running the full DAG on the same file is safe — no duplicate embeddings

MWAA Migration

Q07**Intermediate**

What is AWS MWAA and why are you migrating to it from Docker Compose Airflow?

Hint: Think about what MWAA manages for you vs what you manage yourself.

Key points to cover:

- MWAA (Managed Workflows for Apache Airflow): AWS-managed Airflow service — AWS handles scheduler, webserver, workers, metadata DB, and upgrades
 - Docker Compose Airflow: you manage everything — scheduler crashes, worker scaling, DB backups, version upgrades, security patches
 - MWAA advantages: auto-scaling workers (on-demand), built-in CloudWatch integration, IAM-based auth, VPC isolation, no SSH access needed
 - MWAA metadata DB: Aurora PostgreSQL managed by AWS — no AIRFLOW_DB_URL to configure; MWAA provisions it automatically
 - S3 requirement: MWAA DAG files must live in S3 (s3://bucket/dags/) — no local filesystem DAG loading
 - This is why S3 migration for docs/ is linked to MWAA migration — both require moving off local filesystem
 - IAM roles replace hardcoded credentials: MWAA execution role gets S3 read/write, Neon access via Secrets Manager, ECR pull
 - Cost tradeoff: MWAA is more expensive than self-hosted Compose (~\$200-400/month for small env) but eliminates Airflow ops burden
-

Q08**Advanced**

What specific changes are needed to migrate your four DAGs from Docker Compose Airflow to MWAA? Walk through the migration checklist.

Hint: MWAA has specific constraints that differ from self-hosted Airflow.

Key points to cover:

1. DAG files to S3: copy all DAG Python files to s3://bucket/dags/ — MWAA syncs from S3 automatically every 30s
 2. Replace FileSensor with S3KeySensor: MWAA workers have no access to local filesystem — file watching must use S3KeySensor
 3. requirements.txt to S3: MWAA installs Python dependencies from s3://bucket/requirements.txt — must include langchain, psycpg2-binary, openai, etc.
 4. Airflow connections via Secrets Manager: replace hardcoded connection strings with AWS Secrets Manager references (airflow/connections/neon_db)
 5. Remove AIRFLOW_DB_URL: MWAA manages its own metadata DB — this env var is ignored and irrelevant
 6. IAM execution role: grant MWAA role permissions for S3 (read/write), Secrets Manager (read), ECR (pull), CloudWatch (logs)
 7. VPC configuration: MWAA must be in same VPC as Neon VPC endpoint (or allow Neon's IP ranges) — network ACL update needed
 8. Test with MWAA environment version: MWAA pins Airflow version (2.9 or 2.10) — verify operator imports match that version
-

Q09**Gotcha**

Your collibra_sync_dag runs at 06:00 UTC daily. On MWAA, it fails every Monday with a timeout. How do you diagnose and fix this?

Hint: Think about MWAA worker cold start, Collibra load patterns, and DAG scheduling edge cases.

Key points to cover:

- Diagnosis step 1: check MWAA CloudWatch logs — Worker logs show task start time vs actual Collibra API call time
- Diagnosis step 2: check if Monday timeout correlates with Collibra's weekly batch job (many enterprises run heavy ETL Monday AM)
- Diagnosis step 3: check MWAA worker cold start — if no DAGs ran Sunday night, MWAA may scale down workers; Monday 06:00 triggers a cold start adding 2-3 min latency
- Fix 1: increase task timeout for collibra_sync_dag from default to 30 minutes — allows for Collibra slowness
- Fix 2: add retries=3 with retry_delay=timedelta(minutes=10) — Monday peak passes, retry succeeds
- Fix 3: change schedule to '0 7 * * *' (07:00 UTC) — avoids Monday AM Collibra peak window
- Fix 4: MWAA worker warm-up: schedule a lightweight no-op DAG at 05:55 UTC daily — keeps workers warm before collibra_sync
- Fix 5: implement Collibra API circuit breaker in the sync task — if Collibra is slow, use cached metadata rather than failing

Production Considerations & Design Depth

Q10**Advanced**

Your nightly_refresh_dag runs at 02:00 UTC. What should it actually do and how do you make it idempotent?

Hint: This DAG is referenced but not fully specified — demonstrate you can design it.

Key points to cover:

- Task 1 — refresh_data_product_list: query Databricks for current table list → update DATA_PRODUCTS dict in Airflow Variable → next day's classification uses fresh list
- Task 2 — rebuild_stale_embeddings: find documents in pgvector older than 30 days → re-fetch source from S3 → re-embed → upsert (handles model drift or policy updates)
- Task 3 — prune_old_checkpoints: DELETE FROM checkpoints WHERE created_at < NOW() - INTERVAL '90 days' AND pending_action IS NULL
- Task 4 — warm_cache: run synthetic queries for common intents (data_quality, governance) → pre-populates Redis cache before business hours
- Task 5 — health_report: aggregate metrics (embeddings count, cache hit rate, token usage yesterday) → post to Slack/Teams channel
- Idempotency: rebuild_stale_embeddings uses SHA-256 dedup — re-running produces no duplicates; prune uses a WHERE clause with fixed cutoff — re-running deletes nothing new
- Failure isolation: each task is independent with retries — Task 3 failure doesn't block Task 4
- Backfill safety: nightly_refresh is not backfill-safe by design — running it for yesterday's date would prune today's valid data

Q11**Advanced**

A governance analyst uploads a 500-page PDF policy document via POST /ingest. The embedding step takes 8 minutes and the FastAPI request times out. How do you redesign this?

Hint: Large document ingestion is an async problem — the synchronous HTTP response model doesn't fit.

Key points to cover:

- Root cause: POST /ingest waits for the full pipeline (load → chunk → embed → upsert) synchronously — 500 pages × ~10 chunks × batch=100 = many OpenAI API calls
- Fix 1: fire-and-forget pattern — POST /ingest uploads file to S3, triggers Airflow DAG, immediately returns 202 Accepted + job_id
- Client polls GET /ingest/status/{job_id} for completion — Airflow DAG run status from Airflow REST API
- Fix 2: Airflow DAG as the async worker — all heavy lifting happens in Airflow tasks with proper retries; FastAPI is just the trigger
- Fix 3: chunked progress via SSE — POST /ingest/stream returns SSE events as each stage completes (loaded, chunked, embedded, stored)
- Fix 4: parallel embedding — split chunks into batches, embed in parallel with asyncio.gather or Airflow parallel tasks — reduces 8 min to ~2 min
- Fix 5: pre-signed S3 URL — return a pre-signed URL to the client for direct-to-S3 upload; eliminates FastAPI as a file upload proxy
- MCP tool ingest_document_url already uses fire-and-forget — it triggers the DAG and returns immediately; same pattern should apply to POST /ingest

Q12**Design**

An interviewer asks: 'Why Airflow for document ingestion? Couldn't you just run the ingestion pipeline directly in a FastAPI background task or a Lambda function?' How do you defend your choice?

Hint: This tests whether you can articulate the value of Airflow vs simpler alternatives.

Key points to cover:

- FastAPI background task (asyncio): fine for small documents, but no retry logic, no visibility, no scheduling, process dies on ECS task restart — job lost
- Lambda: 15-minute timeout — 500-page PDF easily exceeds this; cold start adds latency; no built-in workflow orchestration
- Airflow advantages over both: persistent task state (survives restarts), built-in retry with backoff, visual DAG monitoring UI, task-level failure isolation
- Observability: Airflow UI shows exactly which task failed, the error log, how many retries remain — FastAPI background task provides none of this
- Scheduling: collibra_sync_dag and nightly_refresh_dag require reliable cron scheduling — Airflow handles this natively; FastAPI would need APScheduler (another moving part)
- Audit trail: every DAG run is logged with start time, end time, triggered by — important for governance compliance ('when was this policy document ingested?')
- Fair counter-argument: for simple single-document ingestion, Celery or AWS Step Functions would be lighter weight than full Airflow
- Your choice is defensible: project already uses Airflow for scheduling; adding ingestion DAGs reuses existing infrastructure rather than adding a new system